

Mohamed Khider University – Biskra
Computer Science Department

Practical Work N°3

Application of Common Design Patterns

Online Food Ordering System

Module: Software Design
Level: 3rd Year Engineer
Option: Software Engineering
Teacher: Ouaar Hanane
Year: 2025 / 2026

Contents

1	Task 1: Identify Suitable Design Patterns	2
1.1	Objective	2
1.2	Pattern Selection Table	2
1.3	Pattern Categories	2
2	Task 2: Factory Method Pattern – Payment System	4
2.1	Context	4
2.2	Pattern Roles	4
2.3	UML Class Diagram – PlantUML	4
2.4	Java Implementation	4
3	Task 3: Decorator Pattern – Order Add-ons	7
3.1	Context	7
3.2	Pattern Roles	7
3.3	UML Class Diagram – PlantUML	7
3.4	Java Implementation	7
4	Task 4: Observer Pattern – Order Status Notifications	11
4.1	Context	11
4.2	Pattern Roles	11
4.3	UML Class Diagram	11
4.4	UML Sequence Diagram	13
4.5	Java Implementation	14
5	Task 5: Singleton Pattern – OrderManager	17
5.1	Context	17
5.2	Why Singleton is Useful Here	17
5.3	UML Class Diagram – PlantUML	17
5.4	Java Implementation (Thread-Safe)	17
6	Bonus: Why Design Patterns Improve Software Quality	20
6.1	Maintainability	20
6.2	Reusability	20
6.3	Scalability	20

Task 1: Identify Suitable Design Patterns

Objective

For each system requirement, identify the most appropriate design pattern and justify the choice.

Pattern Selection Table

primary		
lightbg Object creation for payment methods	Factory Method	Decouples the creation of payment objects from client code. New payment types can be added by creating a new factory subclass without modifying existing code (Open/-Closed Principle).
Add extra services dynamically	Decorator	Allows behaviour (Extra Cheese, Express Delivery, Gift Wrap) to be added to an order at runtime by wrapping it, rather than creating an exponential number of subclass combinations.
lightbg Notify customers about order status	Observer	Defines a one-to-many dependency so that when the order status changes, all registered observers (customers, admin) are notified automatically without tight coupling.
Ensure only one order manager instance	Singleton	Guarantees a single, globally accessible OrderManager instance, preventing duplicate order IDs, conflicting state, and redundant resource allocation.

Table 1: Design pattern selection for the Online Food Ordering System

Pattern Categories

Creational – Factory Method

Concerned with **how objects are created**. The Factory Method delegates instantiation to subclasses, keeping the client independent of concrete types.

Structural – Decorator

Concerned with **how objects are composed**. The Decorator wraps an existing object to extend its behaviour transparently, without inheritance.

Behavioural – Observer

Concerned with **communication between objects**. The Observer establishes a publish/subscribe mechanism so that state changes propagate automatically.

Creational – Singleton

Concerned with **how objects are created**. The Singleton restricts a class to one instance and provides a global access point to it.

Task 2: Factory Method Pattern – Payment System

Context

The system must support three payment methods: **CreditCardPayment**, **PayPalPayment**, and **CashPayment**. Using the Factory Method pattern, each payment type is created by a dedicated factory subclass, keeping client code decoupled from concrete implementations.

Pattern Roles

primary		
lightbg Product Interface	<code>Payment</code>	Declares <code>processPayment()</code> used by all payment types.
Concrete Product 1	<code>CreditCardPayment</code>	Processes payment via credit card.
lightbg Concrete Product 2	<code>PayPalPayment</code>	Processes payment via PayPal.
Concrete Product 3	<code>CashPayment</code>	Processes payment in cash.
lightbg Creator (Abstract)	<code>PaymentFactory</code>	Declares abstract <code>createPayment()</code> factory method.
Concrete Creator 1	<code>CreditCardFactory</code>	Returns a <code>CreditCardPayment</code> instance.
lightbg Concrete Creator 2	<code>PayPalFactory</code>	Returns a <code>PayPalPayment</code> instance.
Concrete Creator 3	<code>CashFactory</code>	Returns a <code>CashPayment</code> instance.

Table 2: Factory Method roles for the payment subsystem

UML Class Diagram – PlantUML

Java Implementation

```

1  //      Product Interface
2
3  public interface Payment {
4      void processPayment(double amount);
5  }

```

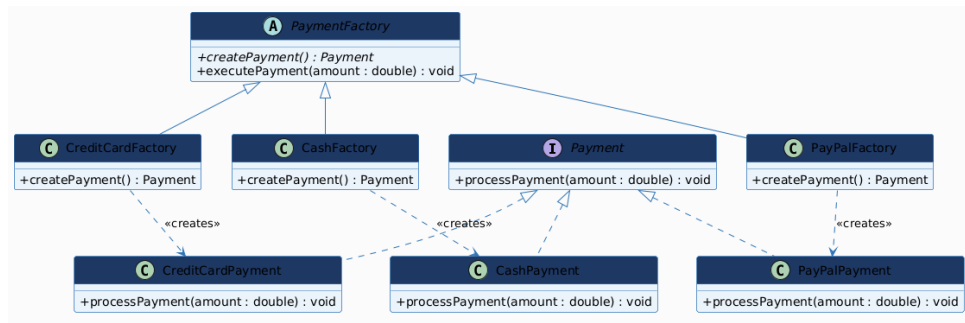


Figure 1: Enter Caption

```

6 //      Concrete Products

7 public class CreditCardPayment implements Payment {
8     @Override
9     public void processPayment(double amount) {
10         System.out.println("Processing credit card payment: $" +
11             amount);
12     }
13 }

14 public class PayPalPayment implements Payment {
15     @Override
16     public void processPayment(double amount) {
17         System.out.println("Processing PayPal payment: $" +
18             amount);
19     }
20 }

21 public class CashPayment implements Payment {
22     @Override
23     public void processPayment(double amount) {
24         System.out.println("Processing cash payment: $" + amount)
25         ;
26     }
27 }

28 //      Abstract Creator

29 public abstract class PaymentFactory {
30     // Factory Method -- subclasses decide which product to
31     // create
32     public abstract Payment createPayment();

33     // Template method that uses the factory method
34     public void executePayment(double amount) {
35         Payment payment = createPayment();
36         payment.processPayment(amount);
  
```

```
37     }
38 }
39
40 //      Concrete Creators
41
42 public class CreditCardFactory extends PaymentFactory {
43     @Override
44     public Payment createPayment() {
45         return new CreditCardPayment();
46     }
47 }
48
49 public class PayPalFactory extends PaymentFactory {
50     @Override
51     public Payment createPayment() {
52         return new PayPalPayment();
53     }
54 }
55
56 public class CashFactory extends PaymentFactory {
57     @Override
58     public Payment createPayment() {
59         return new CashPayment();
60     }
61 }
62
63 //      Client Code
64
65 public class Main {
66     public static void main(String[] args) {
67         PaymentFactory factory = new CreditCardFactory();
68         factory.executePayment(49.99);
69         // Output: Processing credit card payment: $49.99
70
71         factory = new PayPalFactory();
72         factory.executePayment(19.99);
73         // Output: Processing PayPal payment: $19.99
74     }
75 }
```

Listing 1: Factory Method – Full Java Implementation

Task 3: Decorator Pattern – Order Add-ons

Context

An order can be enhanced with optional extras: **ExtraCheese**, **ExpressDelivery**, and **GiftWrap**. These options must be composable and addable at runtime without modifying the base `BasicOrder` class. The Decorator pattern is the ideal solution.

Pattern Roles

primary		
lightbgComponent	<code>FoodOrder</code> (interface)	Declares <code>getDescription()</code> and <code>getCost()</code> for all orders.
Concrete Component	<code>BasicOrder</code>	A plain food order with a name and base price.
lightbgDecorator (Abstract)	<code>OrderDecorator</code>	Holds a <code>FoodOrder</code> reference and delegates calls to it.
Concrete Decorator 1	<code>ExtraCheese</code>	Appends <i>Extra Cheese</i> and adds \$1.50 to cost.
lightbgConcrete Decorator 2	<code>ExpressDelivery</code>	Appends <i>Express Delivery</i> and adds \$5.00 to cost.
Concrete Decorator 3	<code>GiftWrap</code>	Appends <i>Gift Wrap</i> and adds \$2.00 to cost.

Table 3: Decorator roles for the order add-on subsystem

UML Class Diagram – PlantUML

Java Implementation

```

1  //      Component Interface

2  public interface FoodOrder {
3      String getDescription();
4      double getCost();
5  }

6

7  //      Concrete Component

8  public class BasicOrder implements FoodOrder {
9      private String name;

```

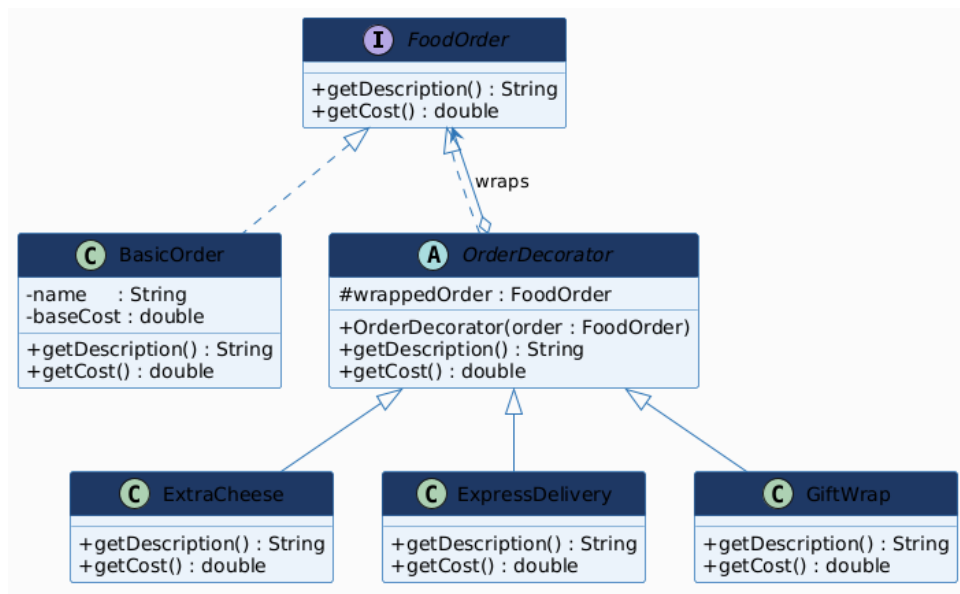



Figure 2: Enter Caption

```

10     private double baseCost;
11
12     public BasicOrder(String name, double baseCost) {
13         this.name = name;
14         this.baseCost = baseCost;
15     }
16
17     @Override
18     public String getDescription() { return name; }
19
20     @Override
21     public double getCost() { return baseCost; }
22 }
23
24 //      Abstract Decorator
25
26 public abstract class OrderDecorator implements FoodOrder {
27     protected FoodOrder wrappedOrder;
28
29     public OrderDecorator(FoodOrder order) {
30         this.wrappedOrder = order;
31     }
32
33     @Override
34     public String getDescription() { return wrappedOrder.
35         getDescription(); }
36
37     @Override
38     public double getCost() { return wrappedOrder.getCost(); }
39 }

```

```
38
39 //      Concrete Decorators

40 public class ExtraCheese extends OrderDecorator {
41     public ExtraCheese(FoodOrder order) { super(order); }
42
43     @Override
44     public String getDescription() {
45         return wrappedOrder.getDescription() + ", Extra Cheese";
46     }
47
48     @Override
49     public double getCost() { return wrappedOrder.getCost() +
50         1.5; }
51 }
52
53 public class ExpressDelivery extends OrderDecorator {
54     public ExpressDelivery(FoodOrder order) { super(order); }
55
56     @Override
57     public String getDescription() {
58         return wrappedOrder.getDescription() + ", Express
59             Delivery";
60     }
61
62     @Override
63     public double getCost() { return wrappedOrder.getCost() +
64         5.0; }
65 }
66
67 public class GiftWrap extends OrderDecorator {
68     public GiftWrap(FoodOrder order) { super(order); }
69
70     @Override
71     public String getDescription() {
72         return wrappedOrder.getDescription() + ", Gift Wrap";
73     }
74
75     @Override
76     public double getCost() { return wrappedOrder.getCost() +
77         2.0; }
78 }
79
80 //      Client Code

81 public class Main {
82     public static void main(String[] args) {
83         FoodOrder order = new BasicOrder("Margherita Pizza",
84             10.0);
```

```
80     order = new ExtraCheese(order);
81     order = new ExpressDelivery(order);
82     order = new GiftWrap(order);
83
84     System.out.println(order.getDescription());
85     // Margherita Pizza, Extra Cheese, Express Delivery, Gift
86       Wrap
87
88     System.out.printf("Total cost: $%.2f\n", order.getCost())
89     ;
90     // Total cost: $18.50
91 }
```

Listing 2: Decorator – Full Java Implementation

Task 4: Observer Pattern – Order Status Notifications

Context

When an order's status changes (**Placed** → **Preparing** → **Delivered**), all interested parties must be notified. The Observer pattern establishes a publish/subscribe relationship so that observers register once and receive every future update automatically.

Pattern Roles

primary		
lightbgSubject (interface)	OrderSubject	Declares <code>attach()</code> , <code>detach()</code> , <code>notifyObservers()</code> .
ConcreteSubject	Order	Stores <code>status</code> ; calls <code>notifyObservers()</code> on every change.
lightbgObserver (interface)	OrderObserver	Declares <code>update(status)</code> called by the subject.
ConcreteObserver 1	Customer	Receives and displays status updates to the end user.
lightbgConcreteObserver 2	AdminDashboard	Logs and aggregates all order status changes centrally.

Table 4: Observer roles for the notification subsystem

UML Class Diagram

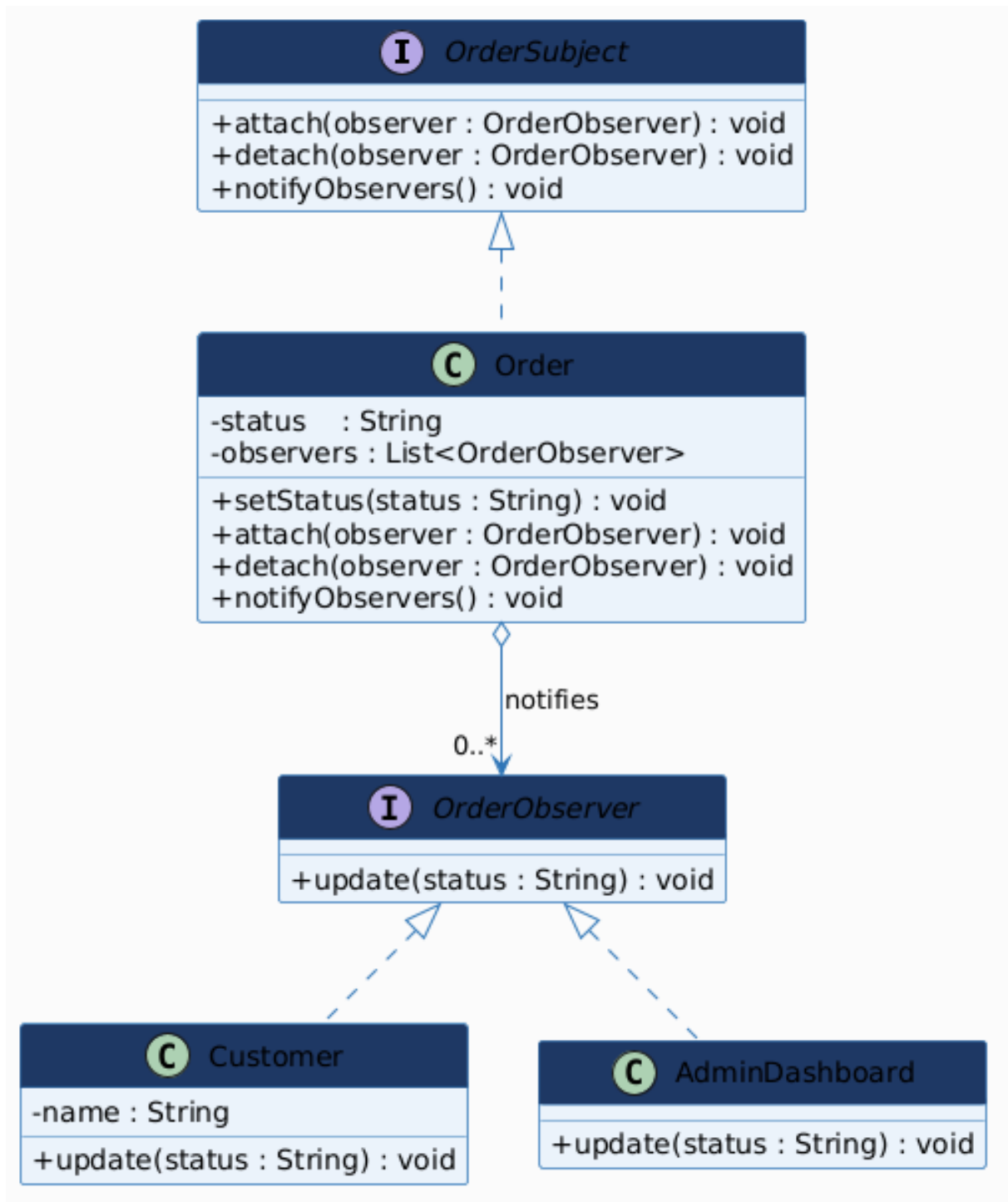


Figure 3: Observer pattern class structure for order notifications

UML Sequence Diagram

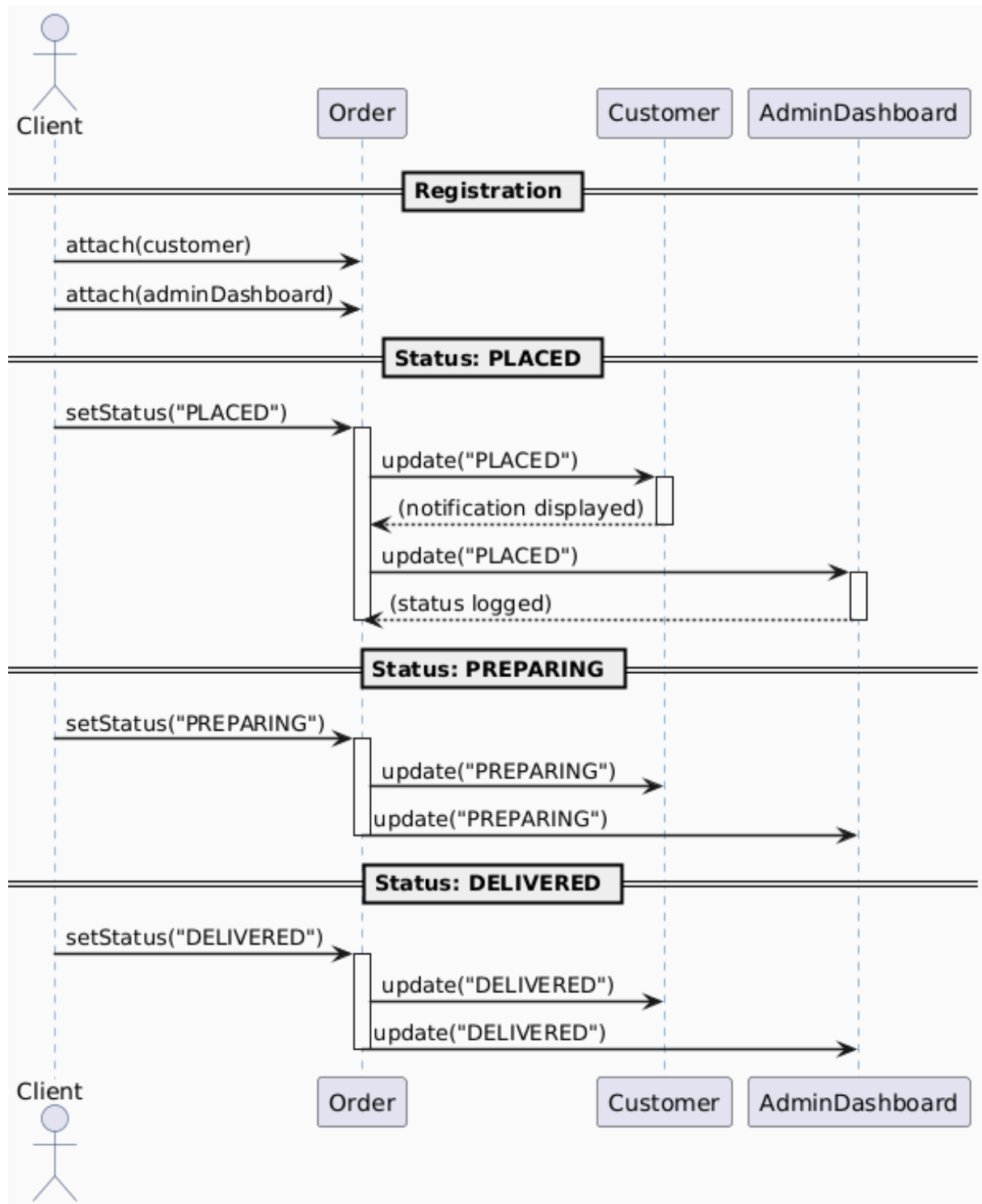


Figure 4: Sequence diagram showing notification flow when order status changes

Java Implementation

```
1 import java.util.ArrayList;
2 import java.util.List;
3
4 //      Observer Interface
5
6 public interface OrderObserver {
7     void update(String status);
8 }
9
10 //      Subject Interface
11
12 public interface OrderSubject {
13     void attach(OrderObserver observer);
14     void detach(OrderObserver observer);
15     void notifyObservers();
16 }
17
18 //      ConcreteSubject
19
20 public class Order implements OrderSubject {
21     private String status;
22     private List<OrderObserver> observers = new ArrayList<>();
23
24     public void setStatus(String status) {
25         this.status = status;
26         System.out.println("[Order] Status changed to: " + status
27             );
28         notifyObservers();
29     }
30
31     @Override
32     public void attach(OrderObserver observer) {
33         observers.add(observer);
34     }
35
36     @Override
37     public void detach(OrderObserver observer) {
38         observers.remove(observer);
39     }
40
41     @Override
42     public void notifyObservers() {
43         for (OrderObserver observer : observers) {
44             observer.update(status);
45         }
46     }
47 }
```

```
44
45 //      ConcreteObservers

46 public class Customer implements OrderObserver {
47     private String name;
48
49     public Customer(String name) { this.name = name; }
50
51     @Override
52     public void update(String status) {
53         System.out.println("    [Customer " + name + "] Your order
54             is now: " + status);
55     }
56 }
57
58 public class AdminDashboard implements OrderObserver {
59     @Override
60     public void update(String status) {
61         System.out.println("    [Admin Dashboard] Order status
62             logged: " + status);
63     }
64 }
65
66 //      Client Code
67
68 public class Main {
69     public static void main(String[] args) {
70         Order order = new Order();
71
72         Customer    alice = new Customer("Alice");
73         AdminDashboard admin = new AdminDashboard();
74
75         order.attach(alice);
76         order.attach(admin);
77
78         order.setStatus("PLACED");
79         order.setStatus("PREPARING");
80         order.setStatus("DELIVERED");
81
82         /*
83         Output:
84         [Order] Status changed to: PLACED
85             [Customer Alice] Your order is now: PLACED
86             [Admin Dashboard] Order status logged: PLACED
87         [Order] Status changed to: PREPARING
88             ...
89         [Order] Status changed to: DELIVERED
90             ...
91         */
92     }
93 }
```


89 }

Listing 3: Observer – Full Java Implementation

Task 5: Singleton Pattern – OrderManager

Context

The system must guarantee that **only one instance** of `OrderManager` exists at runtime. Multiple instances could lead to duplicate order IDs, split order lists, and race conditions. The Singleton pattern solves this by controlling instantiation at the class level.

Why Singleton is Useful Here

- **Centralised tracking:** every module (UI, payment, notifications) accesses the same order list through one shared instance.
- **No duplicate order IDs:** a single ID counter cannot be duplicated if there is only one manager.
- **Consistent state:** operations such as `placeOrder()` and `cancelOrder()` always act on the same data.
- **Resource efficiency:** avoids repeatedly allocating a heavy manager object on every request.

UML Class Diagram – PlantUML

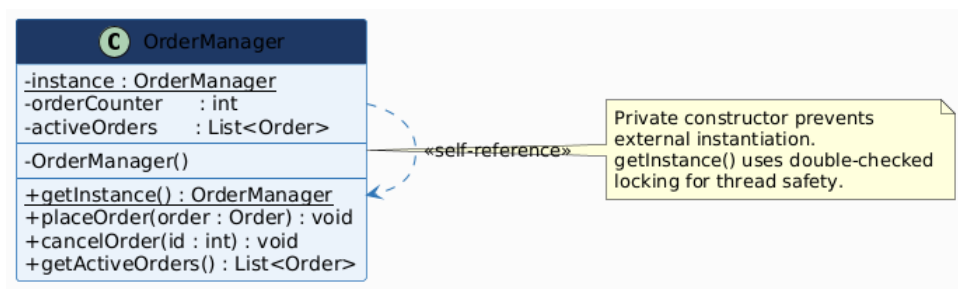


Figure 5: Sequence diagram showing notification flow when order status changes

Java Implementation (Thread-Safe)

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class OrderManager {
5
6     // Step 1: private static volatile instance
7     private static volatile OrderManager instance = null;
8
9     private int          orderCounter = 0;
10    private List<Order> activeOrders = new ArrayList<>();
11
12    // Step 2: private constructor -- no external instantiation
13    private OrderManager() {
  
```

```

14     System.out.println("OrderManager initialised (single
        instance).");
15 }
16
17 // Step 3: public accessor with double-checked locking
18 public static OrderManager getInstance() {
19     if (instance == null) { // 1st check
20         (no lock)
21         synchronized (OrderManager.class) {
22             if (instance == null) { // 2nd check
23                 (with lock)
24                 instance = new OrderManager();
25             }
26         }
27     }
28     return instance;
29 }
30
31 // Business Methods
32
33 public void placeOrder(Order order) {
34     order.setId(++orderCounter);
35     activeOrders.add(order);
36     System.out.println("Order #" + order.getId() + " placed."
37 );
38 }
39
40 public void cancelOrder(int id) {
41     activeOrders.removeIf(o -> o.getId() == id);
42     System.out.println("Order #" + id + " cancelled.");
43 }
44
45 public List<Order> getActiveOrders() {
46     return activeOrders;
47 }
48 }
49
50 // Client Code
51
52 public class Main {
53     public static void main(String[] args) {
54         OrderManager m1 = OrderManager.getInstance();
55         OrderManager m2 = OrderManager.getInstance();
56
57         System.out.println(m1 == m2); // true -- same instance
58
59         m1.placeOrder(new Order("Pizza"));
60         m2.placeOrder(new Order("Burger"));
61     }
62 }

```

```
57         System.out.println(m1.getActiveOrders().size()); // 2
58     }
59 }
```

Listing 4: Singleton – Thread-safe implementation with double-checked locking

Bonus: Why Design Patterns Improve Software Quality

Maintainability

Design patterns enforce a clear **separation of concerns** and provide well-known structures that any developer can recognise immediately. Consider the Factory Method hierarchy: adding a new payment method (e.g. `CryptoPayment`) requires creating *one* new class and *one* new factory — no existing code is touched. This is a direct application of the **Open/Closed Principle** (open for extension, closed for modification).

The Observer pattern similarly isolates the notification logic: changing how a customer is alerted (e.g. switching from console output to email) only affects the `Customer` class, not the `Order` or any other observer.

Maintainability – Key benefits

- Predictable code structure reduces the time needed to understand unfamiliar modules.
- Changes are localised to a single class or hierarchy.
- Less risk of regressions when adding features.

Reusability

Patterns encourage **programming to interfaces** rather than concrete implementations. The `Payment` interface, the `FoodOrder` component interface, and the `OrderObserver` interface are fully decoupled from their implementations. This means:

- The `PayPalPayment` class could be extracted into a shared library and reused by a subscription billing module.
- The `OrderDecorator` hierarchy is generic enough to be adapted for any wrappable object (e.g. product configurators, middleware pipelines).
- The `OrderManager` Singleton can be extended with persistence or caching without changing callers.

Reusability – Key benefits

- Interfaces allow the same pattern structure to be dropped into new projects.
- Concrete classes depend on abstractions, not on each other.
- Reduces code duplication across teams and modules.

Scalability

The system can grow **horizontally** without architectural changes:

- **Factory Method:** a new restaurant partner with a unique payment gateway requires only a new `PaymentFactory` subclass.
- **Decorator:** a new order option (e.g. *Priority Packaging*) is one new decorator class; all existing combinations continue to work.

- **Observer:** a new subscriber (e.g. a delivery driver app) registers itself without any change to `Order`.
- **Singleton:** the `OrderManager` can be evolved into a distributed service (e.g. backed by a database) without changing client code, because all access goes through `getInstance()`.

Scalability – Key benefits

- Each pattern grows independently; adding to one does not force changes in others.
- The single-entry-point model (Factory, Singleton) makes horizontal scaling straightforward.
- The system handles increasing feature complexity without becoming monolithic.

End of Practical Work N°3 · Software Design · 2025–2026